

Laboratorio 0: Práctica de Calentamiento

- Lógica de Programación, Algoritmos y Estructuras de Datos en Java

Metadatos

- **Tiempo estimado de desarrollo:** 3 horas
 - **Herramientas requeridas:**
 - Kit de Desarrollo de Java (JDK) versión 17 o superior.
 - Entorno de Desarrollo Integrado (VS Code con el "Extension Pack for Java", IntelliJ IDEA o Eclipse).
 - Consola o terminal de comandos para compilar y ejecutar programas.
 - **Metas de aprendizaje:**
 - **Aplicar los conceptos fundamentales de Programación Orientada a Objetos (POO)** en Java mediante el diseño de clases cohesivas, encapsulamiento y manipulación de constructores.
 - **Implementar algoritmos de procesamiento de datos y colecciones** utilizando bucles de control, condicionales y clases de la biblioteca estándar de Java (`List`, `ArrayList`, `LocalDateTime`).
 - **Identificar, diagnosticar y corregir errores comunes de ejecución en Java** (tales como `NullPointerException`, `IndexOutOfBoundsException` e inconsistencias en la comparación de referencias).
-

Parte 1 (Práctica Guiada)

Planteamiento del problema

En el ámbito empresarial, los sistemas informáticos a menudo deben procesar grandes lotes de transacciones financieras. El objetivo de este ejercicio es implementar un analizador lógico que procese un historial de transacciones bancarias en memoria, filtre registros válidos y genere estadísticas clave (como saldos netos y promedios) utilizando estructuras algorítmicas clásicas en Java.

Usted construirá una pequeña aplicación de consola dividida en tres clases: 1. `Transaccion`: Representa la entidad de datos transaccionales. 2. `AnalizadorTransacciones`: Contiene los algoritmos de filtrado, sumatorias y cálculos estadísticos. 3. `Main`: Inicializa el lote de pruebas y ejecuta el flujo principal del programa.

Estructura de archivos y código del proyecto

1. [NEW] `Transaccion.java`

Cree este archivo para definir la estructura de datos que modela una transacción financiera. Observe el uso correcto de tipos de datos, encapsulamiento de propiedades mediante modificadores `private` y la API de fechas moderna de Java (`java.time`):

```
package laboratorio0;  
  
import java.time.LocalDateTime;  
  
public class Transaccion {  
    private String id;  
    private double monto;  
    private String tipo; // "DEBITO" o "CREDITO"  
    private String cuentaOrigen;  
    private LocalDateTime fecha;  
  
    // Constructor parametrizado  
    public Transaccion(String id, double monto, String tipo,  
                        String cuentaOrigen, LocalDateTime fecha) {  
        this.id = id;  
        this.monto = monto;  
        this.tipo = tipo;  
        this.cuentaOrigen = cuentaOrigen;  
        this.fecha = fecha;  
    }  
}
```

```

    // Métodos accesorios (Getters)
    public String getId() {
        return id;
    }

    public double getMonto() {
        return monto;
    }

    public String getTipo() {
        return tipo;
    }

    public String getCuentaOrigen() {
        return cuentaOrigen;
    }

    public LocalDateTime getFecha() {
        return fecha;
    }

    // Sobrescritura de toString para facilitar la visualización del objeto
    @Override
    public String toString() {
        return String.format(
            "Transaccion[ID=%s, Monto=%.2f, Tipo=%s, Cuenta=%s, Fecha=%s]",
            id, monto, tipo, cuentaOrigen, fecha);
    }
}

```

2. [NEW] AnalizadorTransacciones.java

Cree este archivo e implemente los algoritmos de análisis. En este componente usted implementará la lógica para procesar listas dinámicas, realizar acumulaciones de valores de punto flotante y filtrar objetos en base a criterios condicionales:

```

package laboratorio0;

import java.util.ArrayList;
import java.util.List;

public class AnalizadorTransacciones {

    /**

```

```

    * Calcula el saldo neto acumulado del lote de transacciones.
    * Los créditos aumentan el saldo (+) y los débitos lo disminuyen (-).
    * @param transacciones Lista de transacciones a procesar.
    * @return El saldo neto calculado.
    */
    public double calcularSaldoNeto(List<Transaccion> transacciones) {
        if (transacciones == null || transacciones.isEmpty()) {
            return 0.0;
        }

        double saldoNeto = 0.0;
        for (Transaccion t : transacciones) {
            // Validación defensiva para evitar llamadas sobre valores nulos
            if (t.getTipo() != null) {
                if ("CREDITO".equalsIgnoreCase(t.getTipo())) {
                    saldoNeto += t.getMonto();
                } else if ("DEBITO".equalsIgnoreCase(t.getTipo())) {
                    saldoNeto -= t.getMonto();
                }
            }
        }
        return saldoNeto;
    }

    /**
     * Filtra y retorna las transacciones cuyo monto supera un umbral establecido.
     * @param transacciones Lista original de transacciones.
     * @param umbral Monto límite exclusivo.
     * @return Lista filtrada de transacciones.
     */
    public List<Transaccion> obtenerTransaccionesDeAltoValor(
        List<Transaccion> transacciones, double umbral) {
        List<Transaccion> altoValor = new ArrayList<>();
        if (transacciones == null) {
            return altoValor;
        }

        for (Transaccion t : transacciones) {
            if (t.getMonto() > umbral) {
                altoValor.add(t);
            }
        }
        return altoValor;
    }

    /**

```

```

    * Calcula el promedio de monto para un tipo específico de transacción.
    * @param transacciones Lista original.
    * @param tipo Tipo de transacción a promediar ("DEBITO" o "CREDITO").
    * @return Promedio aritmético de los montos, o 0.0 si no hay coincidencias.
    */
    public double calcularPromedioPorTipo(
        List<Transaccion> transacciones, String tipo) {
        if (transacciones == null || transacciones.isEmpty() || tipo == null) {
            return 0.0;
        }

        double suma = 0.0;
        int contador = 0;
        for (Transaccion t : transacciones) {
            if (tipo.equalsIgnoreCase(t.getTipo())) {
                suma += t.getMonto();
                contador++;
            }
        }

        // Evitar división por cero
        return contador > 0 ? (suma / contador) : 0.0;
    }
}

```

3. [NEW] Main.java

Cree la clase contenedora del método de ejecución inicial. Aquí se instanciarán los datos de prueba simulados y se invocará al analizador lógico para imprimir la salida esperada en la consola:

```

package laboratorio0;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Transaccion> historial = new ArrayList<>();

        // Carga de historial de pruebas (con marcas temporales simuladas)
        historial.add(new Transaccion("T01", 1500.00, "CREDITO",
            "CTA-101", LocalDateTime.now().minusHours(5)));
        historial.add(new Transaccion("T02", 200.00, "DEBITO",

```

```

        "CTA-101", LocalDateTime.now().minusHours(4)));
    historial.add(new Transaccion("T03", 450.50, "DEBITO",
        "CTA-102", LocalDateTime.now().minusHours(3)));
    historial.add(new Transaccion("T04", 3000.00, "CREDITO",
        "CTA-103", LocalDateTime.now().minusHours(2)));
    historial.add(new Transaccion("T05", 120.00, "DEBITO",
        "CTA-101", LocalDateTime.now().minusHours(1)));
}

AnalizadorTransacciones analizador = new AnalizadorTransacciones();

// Invocar algoritmos de procesamiento lógico
double saldoNeto = analizador.calcularSaldoNeto(historial);
double promedioDebitos = analizador.calcularPromedioPorTipo(historial, "DEBITO");
List<Transaccion> transaccionesGrandes =
    analizador.obtenerTransaccionesDeAltoValor(historial, 1000.0);

// Imprimir reporte de resultados en la consola estándar
System.out.println("=== INFORME DE ANÁLISIS FINANCIERO ===");
System.out.printf("Saldo Neto Acumulado: %.2f CRC\n", saldoNeto);
System.out.printf("Monto Promedio de Débitos: %.2f CRC\n", promedioDebitos);

System.out.println("\nTransacciones de alto valor (> 1000.00 CRC):");
for (Transaccion t : transaccionesGrandes) {
    System.out.println(" - " + t);
}
}
}

```

Captura y salida esperada en consola

Al compilar y ejecutar el programa desde su entorno o terminal, usted debe visualizar el siguiente reporte en pantalla:

```

=== INFORME DE ANÁLISIS FINANCIERO ===
Saldo Neto Acumulado: 3729,50 CRC
Monto Promedio de Débitos: 156,83 CRC

Transacciones de alto valor (> 1000.00 CRC):
- Transaccion[ID=T01, Monto=1500,00, Tipo=CREDITO, Cuenta=CTA-101, Fecha=2026-08-10T10:20:46...]
- Transaccion[ID=T04, Monto=3000,00, Tipo=CREDITO, Cuenta=CTA-103, Fecha=2026-08-10T13:20:46...]

```

(Nota: Las fechas impresas variarán según la hora exacta en la que ejecute la prueba).

Parte 2 (Depuración)

Planteamiento del error común

Un programador junior ha intentado escribir un método rápido para generar reportes sobre cuentas particulares y detectar egresos. Su clase se llama `ProcesadorReportes.java`. Al ejecutar su lógica sobre una lista de transacciones reales, el programa aborta de forma abrupta lanzando una excepción a la consola y omitiendo registros importantes.

Analice detenidamente el código fuente problemático planteado a continuación:

Código con fallas (`ProcesadorReportes.java`)

```
package laboratorio0;  
  
import java.util.List;  
  
public class ProcesadorReportes {  
  
    public static void generarReporteCuenta(  
        List<Transaccion> transacciones, String cuentaBuscada) {  
        System.out.println("Generando reporte para la cuenta: " + cuentaBuscada);  
  
        // Iterar el historial de transacciones  
        for (int i = 0; i <= transacciones.size(); i++) {  
            Transaccion t = transacciones.get(i);  
  
            // Comparar si la transacción pertenece a la cuenta buscada  
            if (t.getCuentaOrigen() == cuentaBuscada) {  
  
                // Imprimir según sea ingreso o egreso  
                if (t.getTipo().toUpperCase().equals("DEBITO")) {  
                    System.out.printf("Egreso detectado - Monto: %.2f\n", t.getMonto());  
                } else {  
                    System.out.printf("Ingreso detectado - Monto: %.2f\n", t.getMonto());  
                }  
            }  
        }  
    }  
}
```

Guía instruccional para analizar el error en consola

Ejecute el análisis mental o configure el código en su editor y siga los siguientes pasos de diagnóstico técnico:

1. **Paso 1 (Localizar error de límites de arreglo):** Cuando ejecute este código, la aplicación fallará al final del ciclo for con la siguiente excepción: `text Exception in thread "main"`
`java.lang.IndexOutOfBoundsException: Index 5 out of bounds for length 5 at`
`java.base/java.util.ArrayList.rangeCheck(ArrayList.java:187) at`
`java.base/java.util.ArrayList.get(ArrayList.java:435) at`
`laboratorio0.ProcesadorReportes.generarReporteCuenta(ProcesadorReportes.java:12)`
Diagnóstico: En Java, los arreglos y listas son indexados a partir de 0 hasta `size() - 1`. En la condición del bucle `i <= transacciones.size()`, cuando `i` alcanza el tamaño de la lista (en este caso 5), el método `transacciones.get(5)` intenta acceder a una posición fuera de los límites de la colección, provocando que el programa caiga.
2. **Paso 2 (Localizar error lógico de cadenas):** Note la comparación de strings:
`t.getCuentaOrigen() == cuentaBuscada.` **Diagnóstico:** En Java, el operador `==` compara la **identidad** de las referencias de memoria de los objetos (si apuntan exactamente a la misma dirección física) y no su **contenido**. Si una cadena de texto es generada de manera dinámica en tiempo de ejecución (por ejemplo, leída desde un archivo, base de datos o por entrada del teclado), su dirección de memoria será diferente a la del literal del parámetro `cuentaBuscada`, causando que la condición retorne `false` de manera silenciosa, omitiendo transacciones válidas.
3. **Paso 3 (Diagnosticar NullPointerException silencioso):** Examine la línea
`t.getTipo().toUpperCase().` **Diagnóstico:** Si por error o diseño una transacción es registrada en el sistema con el valor del atributo `tipo` en `null` (por ejemplo, transacciones de depósitos pendientes de clasificación), al intentar ejecutar el método `.toUpperCase()` sobre una referencia nula, Java disparará inmediatamente un error fatal de tipo `java.lang.NullPointerException`.

Procedimiento exacto para resolverlo

Para solucionar todas las fallas descritas anteriormente, aplique las siguientes correcciones de refactorización:

1. **Corrección de límites:** Cambie la condición de iteración en el bucle for para usar `<` en lugar de `<=`.

2. **Corrección de comparación de cadenas:** Reemplace el operador `==` por el método estándar `.equals()` (o `.equalsIgnoreCase()` para evitar discrepancias de mayúsculas).
3. **Control de referencias nulas (NPE):** Al comparar una cadena constante contra el valor de una variable que podría ser nula, coloque la constante primero. Por ejemplo:
`"DEBITO".equalsIgnoreCase(t.getTipo())`. Al hacerlo así, si `t.getTipo()` es null, el método `.equalsIgnoreCase` manejará el valor de manera segura retornando false en lugar de lanzar una excepción de puntero nulo.

A continuación se muestra el código corregido y robusto:

```
package laboratorio0;  
  
import java.util.List;  
  
public class ProcesadorReportes {  
  
    public static void generarReporteCuenta(  
        List<Transaccion> transacciones, String cuentaBuscada) {  
        // Validación de parámetros de entrada  
        if (transacciones == null || cuentaBuscada == null) {  
            System.out.println("Entrada no válida para el reporte.");  
            return;  
        }  
  
        System.out.println("Generando reporte para la cuenta: " + cuentaBuscada);  
  
        // Corrección de límites: i < transacciones.size()  
        for (int i = 0; i < transacciones.size(); i++) {  
            Transaccion t = transacciones.get(i);  
  
            // Programación defensiva ante elementos nulos en la lista  
            if (t == null) continue;  
  
            // Corrección de comparación: equals() en lugar de ==  
            if (cuentaBuscada.equalsIgnoreCase(t.getCuentaOrigen())) {  
  
                // Evitar NPE colocando la constante al inicio  
                if ("DEBITO".equalsIgnoreCase(t.getTipo())) {  
                    System.out.printf("Egreso detectado - Monto: %.2f CRC\n", t.getMonto());  
                } else {  
                    System.out.printf("Ingreso detectado - Monto: %.2f CRC\n", t.getMonto());  
                }  
            }  
        }  
    }  
}
```

```
    }  
  }  
}
```

Parte 3 (Reto Autónomo)

Especificaciones técnicas a cumplir

Para evaluar sus habilidades analíticas y de resolución de problemas bajo restricciones algorítmicas, se le solicita extender la solución e implementar un **Algoritmo de Detección de Fraude**.

Deberá crear una nueva clase llamada `DetectorFraude` en la cual implementará un método encargado de analizar el historial transaccional en busca de dos patrones de riesgo particulares:

- **[] Patrón de Frecuencia Sospechosa:** Dos transacciones consecutivas de la **misma cuenta de origen** que ocurran en un lapso de tiempo **menor o igual a 60 segundos** (1 minuto), donde al menos una de las transacciones sea de tipo `DEBITO`.
- **[] Patrón de Monto Anómalo:** Cualquier transacción individual cuyo monto sea estrictamente superior a **5 veces el monto promedio** de las transacciones del tipo `DEBITO` calculadas para esa misma cuenta de origen en el lote completo.

Firma del método requerida:

```
package laboratorio0;  
  
import java.util.List;  
  
public class DetectorFraude {  
    /**  
     * Analiza el lote de transacciones e identifica aquellas que cumplen con patrones sospechosos.  
     * @param transacciones Historial completo a analizar.  
     * @return Lista de transacciones clasificadas como sospechosas.  
     */  
    public List<Transaccion> analizarComportamientoSospechoso(  
        List<Transaccion> transacciones) {
```

```
// Implemente su algoritmo aquí  
}  
}
```

Restricciones e indicaciones algorítmicas:

1. Para calcular la diferencia de tiempo en segundos entre dos marcas temporales `LocalDateTime`, usted puede utilizar el método `java.time.Duration.between(inicio, fin).getSeconds()`. Recuerde usar el valor absoluto si no está seguro del orden de los registros, o garantizar que analiza la lista en orden cronológico.
2. Al diseñar el algoritmo para el *Monto Anómalo*, deberá calcular primero el promedio de débitos correspondientes a esa cuenta origen específica y luego evaluar la regla de umbral de 5x.
3. Asegúrese de que su algoritmo no lance excepciones si recibe una lista vacía o nula, en su lugar debe retornar una colección vacía.

Cómo verificar el éxito de su solución

1. Modifique la clase `Main` y configure un lote de transacciones de prueba donde ocurran dos transacciones de la misma cuenta de origen con 30 segundos de diferencia.
2. Agregue también una transacción de débito con un monto de `5000.00` para una cuenta cuyo promedio de débito general sea de `100.00`.
3. Invoque su clase `DetectorFraude` pasándole este lote de pruebas y compruebe que el reporte de alertas de salida imprima exactamente las transacciones implicadas en las alertas.
4. Asegúrese de que ninguna otra transacción normal sea detectada falsamente por las condiciones del algoritmo.